ТЕМА 4. АНИМАЦИЯ

ЛЕКЦИЯ 4.1. СВОЙСТВО TRANSFORM И CSS ПЕРЕХОДЫ.

Свойство transform. CSS-переходы. Свойство transition.

Свойство transform

Модель визуального форматирования CSS описывает систему координат для каждого позиционированного элемента.

Мы уже взаимодействовали с ней, когда задавали элементам позиционирование с параметрами (top, bottom, left, right) или настраивали **z-index**.

Но такая система существует и внутри элемента. Именно она является точкой отсчета для свойств смещения. Положение и размеры в этом координатном пространстве можно рассматривать как заданные в пикселях, относительно точки отсчета, с положительными значениями, идущими вправо и вниз. Это координатное пространство можно изменить с помощью свойства **transform**.

CSS-свойство **transform** позволяет вам поворачивать, масштабировать, наклонять или сдвигать элемент.

Трансформации преобразовывают элемент, не затрагивая остальные элементы веб-страницы, т.е. другие элементы не сдвигаются относительно него.

Редактировать можно только трансформируемые элементы. То есть все элементы, макет которых определяется блочной моделью CSS, за исключением: inline-блоков, элементов col и colgroup.

Значение свойства задается в виде функций, описывающих процесс преобразования элемента. Аргументом функции

©Pechora-PRO

может быть количество пикселей и значение градуса или число.

transform-origin

С помощью этого свойства можно изменить точку отсчета преобразований. По умолчанию, она находится в центре объекта, но может быть смещена ключевыми словами (top, bottom, left, right) или числовыми значениями.

transform-style

При настройке свойств, связанных с положением элемента в трехмерном пространстве, могут возникнуть сложности в демонстрации изменений. Для определения, будут ли дочерние элементы элемента расположены в трехмерном пространстве или «сплющены» в плоскости элемента используют свойство **transform-style**.

Возможными значениями свойства будут **flat** - дочерние элементы лежат в плоскости самого элемента и **preserve-3d** - дочерние элементы должны располагаться в 3D-пространстве.

```
<body>
    <section id="example-element">
        <div class="face front">1</div>
        <div class="face back">2</div>
        <div class="face right">3</div>
        <div class="face left">4</div>
        <div class="face top">5</div>
        <div class="face bottom">6</div>
    </section>
</body>
```

```
transform-style
```






```
flat;
```

```
preserve-3d;
```


©Pechora-PRO

Сплющенные дочерние элементы не будут существовать сами по себе в трехмерном пространстве. Поскольку это свойство не наследуется, оно должно быть установлено для всех неконечных потомков элемента.

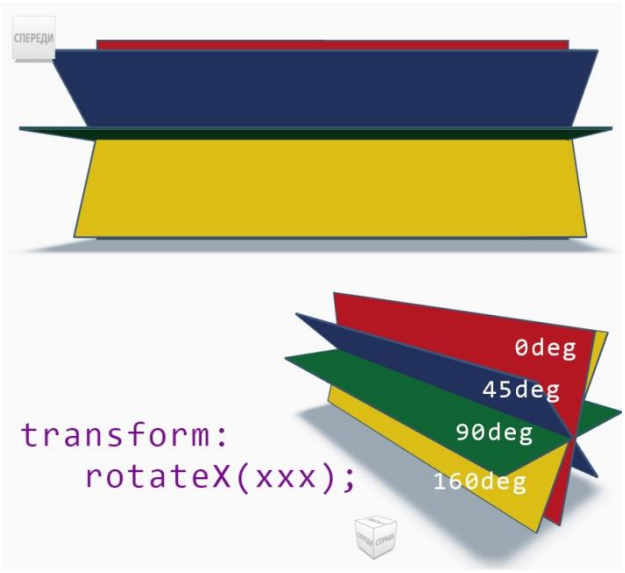
Вращение

- **rotate(a)** Вращает элемент вокруг фиксированной точки на 2D-плоскости. Задаем угол поворота: положительный, движение будет по часовой стрелке; отрицательный - против часовой стрелки.

```
rotate(0);  rotate(45deg); rotate(-90deg); rotate(180deg);
```



- **rotateX(a)** Поворачивает элемент вокруг горизонтальной оси. rotateX(a) эквивалентно rotate3d(1, 0, 0, a).




```
rotatex(45deg);
```



```
rotatex(20deg);
```



```
rotatex(180deg);
```

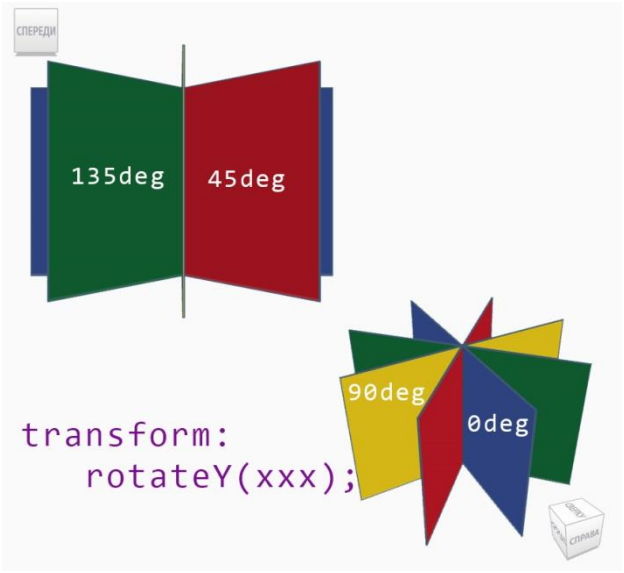


```
rotatex(300deg);
```



- **rotateY(a)** Поворачивает элемент вокруг вертикальной оси. rotateY(a) эквивалентно rotate3d(0, 1, 0, a)

©Pechora-PRO



rotatey(20deg);



rotatey(45deg);



rotatey(180deg);



rotatey(300deg);



- **rotateZ(a)** Вращает элемент вокруг оси Z. rotateZ(a) эквивалентно rotate(a) или rotate3d(0, 0, 1, a).
- **rotate3d(x,y,z,a)** Вращает элемент вокруг фиксированной оси в 3D-пространстве. Параметры вращения, определяются тремя координатами x, y и z вектора, обозначающего ось вращения и углом поворота. Угол поворота: положительный, движение будет по часовой стрелке; отрицательный - против часовой стрелки.

```
rotate3d(1, 1, 1, -20deg);
```



```
rotate3d(1, 1, 0, 45deg);
```



```
rotate3d(0, 1, 1, 45deg);
```



```
rotate3d(1, 0, 1, 60deg);
```



©Pechora-PRO

Масштабирование (изменение размера)

Свойство задается координатами, которые определяют степень масштабирования в каждом направлении. Если все **координаты равны**, масштабирование равномерное и соотношение сторон элемента сохраняется.

Когда значение **координаты** выходит **за пределы диапазона** [-1, 1], элемент увеличивается по этому измерению; когда находится внутри - сжимается.

Если **значение отрицательное**, результатом будет отражение элемента в этом измерении.

Значение **1** не влияет на масштаб.

- **scale(s)** Масштабирует элемент по горизонтали и/или вертикали на 2D-плоскости.

```
scale(1.2);
```



```
scale(1.2, 0.5);
```



```
scale(0.5);
```



```
scale(-0.5 1);
```



©Pechora-PRO

- **scaleX(sx)** Масштабирует элемент по горизонтали. Поведение эквивалентно scale(sx, 1) или scale3d(sx, 1, 1).

```
scaleX(1.0);
```



```
scaleX(1.3);
```



```
scaleX(0.5);
```



```
scaleX(-1.0);
```



- **scaleY(sy)** Масштабирует элемент по вертикали. scaleY(sy) эквивалентно scale(1, sy) или scale3d(1, sy, 1). transform: rotateX(180deg);===transform: scaleY(-1);

©Pechora-PRO

`scaleY(1.0);`



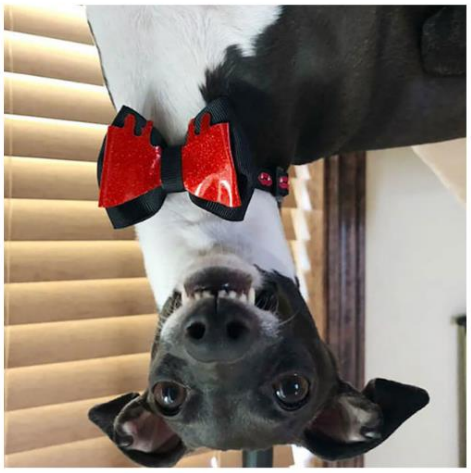
`scaleY(1.3);`



`scaleY(0.5);`



`scaleY(-1.0);`



- **scaleZ(sz)** Масштабирует элемент по оси Z. Это масштабное преобразование изменяет z-координату каждой точки элемента на постоянный коэффициент, за исключением случаев, когда sz=1, и в этом случае функция представляет собой тождественное преобразование. Масштабирование не изотропно, и углы элемента не сохраняются.

©Pechora-PRO

Важно! Эффект будет заметен только при использовании перспективы, поворота элемента по осям X или Y, а также при смещении элемента по оси Z.

```
rotate3d(1, 1, 0, 45deg) + scaleZ()
```

```
scaleZ(0.5);
```



```
scaleZ(1.5);
```



- **scale3d(sx, sy, sz),** масштабирует элемент в 3D-пространстве. Это масштабное преобразование характеризуется трехмерным вектором.

```
rotate3d(1, 1, 0, 45deg) + scale3d()
```

```
scale3d(1, 0.2, 0.5)
```



```
scale3d(1, 1, 0.5);
```



Искажение

- **skew()** Наклоняет элемент на 2D-плоскости.

```
skew(10deg, 10deg);
```

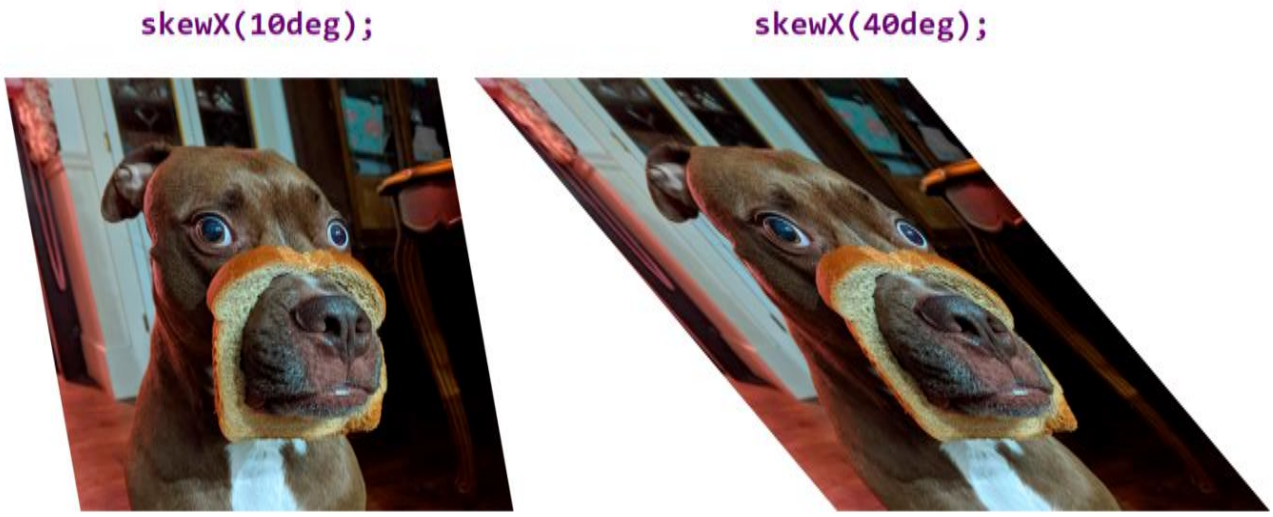


```
skew(-10deg, -10deg);
```



©Pechora-PRO

- **skewX(a)** Наклоняет элемент в горизонтальном направлении.



- **skewY(a)** Наклоняет элемент в вертикальном направлении.



©Pechora-PRO

Перемещение

- **translate()** Переносит элемент на 2D-плоскости.

```
translate(30px);
```



```
translate(0px, 20px);
```



```
translate(-10px, -10px);
```



```
translate(30px, -20px);
```



- **translateX(tx)** Перемещает элемент по горизонтали.

```
translateX(100px);
```

```
translateX(-100px);
```

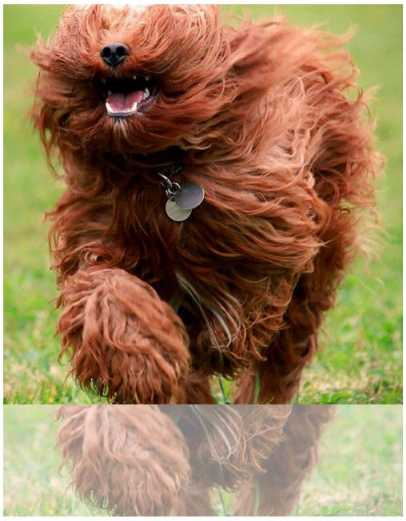


©Pechora-PRO

- **translateY(ty)** Перемещает элемент по вертикали.

```
translateY(70px);
```





```
translateY(-70px);
```

- **translateZ(tz)** Перемещает элемент по оси Z.

```
translateZ(0px);
```

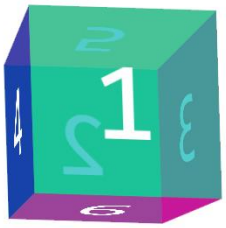


```
translateZ(70px);
```



- **translate3d(tx, ty, tz)** Изменяет положение элемента в трехмерном пространстве.

```
translate3d(0px, 0px, 0px)
```



```
translate3d(42px, -62px, -135px)
```



Перспектива

©Pechora-PRO

perspective() Устанавливает расстояние между пользователем и плоскостью z=0.

Значения, отличные от **none** заставляют элементы с положительными позициями по оси z казаться больше, а элементы с отрицательными позициями по z - меньше. Элементы с позициями по z, равными или превышающими значение перспективы, исчезают, как если бы они находились позади пользователя. Отрицательные значения являются синтаксическими ошибками.

translateZ(100px) perspective(400px)



translateZ(100px) perspective(100px)



ranslateZ(100px) perspective(200px)



Матричное преобразование

- **matrix()** Описывает однородную матрицу двумерного преобразования. Функция задается шестью значениями. Значения представляют следующие функции: scaleX(), skewY(), skewX(), scaleY(), translateX(), translateY().
- **matrix3d()** Описывает трехмерное преобразование как однородную матрицу 4×4. Здесь значений уже 16. Фактически matrix3d() является альтернативой функциям трехмерного преобразования rotate3d(), rotateX(), rotateY(), rotateZ(), translate3d(), translateZ(), scale3d(), scaleZ(), и perspective().

©Pechora-PRO

В текущем курсе не будем углубляться в процесс формирования матриц. Можете посмотреть, какие значения формируются для матрицы на тренажере – http://www.useragentman.com/matrix/

CSS-переходы

CSS переходы предоставляют способ контролировать скорость анимации, при изменении CSS-свойств. Вместо того чтобы свойство применилось сразу, вы можете сделать это действие происходящим в течение какого-то момента времени.

Например, если вы настроите смену цвета элемента с белого на красный при наведении, изменение произойдёт моментально. А вот с CSS переходами, изменения произойдут за некоторый временной интервал, настроенный с помощью функции.

Анимации, составляющие переход между двумя состояниями, часто называют **неявными переходами**, так как промежуточные состояния в период между начальным и конечным состоянием неявно определяются браузером.

CSS переходы позволяют вам решить:

- какие свойства нужно анимировать (перечислением их явно);
- когда анимация начнётся (установкой задержки);
- как долго переход будет выполняться (установкой продолжительности);
- как она будет запущена (определением функции по времени, например, линейно или быстро в начале, медленно в конце).

Разработчик может определить какое свойство и как анимировать. Это позволяет создавать сложные переходы. Так как некоторые свойства не имеет смысла анимировать, перечень доступных для анимирования свойств ограничен

©Pechora-PRO

определённым набором: изменение цвета, границ, теней, вариации transform и т.д.

Свойство transition

CSS переходы контролируются свойством **transition**. Это лучший способ их настроить, позволяет избежать длинных свойств и их несоответствий, которые могут сделать отладку CSS долгой.

Можно контролировать определённые параметры перехода следующими подсвойствами.

transition: <property> <duration> <timing-function> <delay>;

transition-property

Указывает имя или имена свойств, чьи переходы должны анимироваться. Только свойства, указанные здесь, анимируются в переходах; изменение других свойств будет происходить обычным образом.

transition-duration

Определяет время происхождения перехода. Можно указать время анимирования всех свойств перехода сразу или для каждого свойства в отдельности.

transition-timing-function

С помощью функций задаем плавность перехода между состояниями элемента.

- **ease** - значение по умолчанию, скорость увеличивается к середине перехода, замедляясь в конце.
- **linear** - переходы с неизменной скоростью.
- **ease-in** – переход начинается медленно, с увеличением скорости до полного завершения.
- **ease-out** - переход начинается быстро, с замедлением при завершении.

©Pechora-PRO

- **ease-in-out** - начинается медленно, ускоряется, а затем снова замедляется.
- **cubic-bezier(p1, p2, p3, p4)** - кубическая кривая Безье, где значения p1 и p3 должны находиться в диапазоне от 0 до 1.
- **steps(n, <jumpterm>)** - переход с **n** остановок во время перехода, отображая каждую остановку в течение равных промежутков времени. Например, если n равно 5, есть 5 шагов.
    - **jump-start | start** обозначает непрерывную функцию, так что первый скачок происходит, когда начинается переход;
    - **jump-end | end** обозначает непрерывную функцию, так что последний переход происходит, когда анимация заканчивается;
    - **jump-none** прыжка с обоих концов нет. Вместо этого, переход удерживается на отметке 0% и отметке 100%, в течение 1/n продолжительности перехода.
    - **jump-both** включает паузы на отметках 0% и 100%, эффективно добавляя шаг во время перехода.
    - **step-start** эквивалентно steps(1, jump-start)
    - **step-end** эквивалентно steps(1, jump-end)

transition-delay

Определяет задержку перед началом перехода к новому состоянию.

©Pechora-PRO